

פרויקט גמר · הנדסאי תוכנה · מגמת DevOps

ספר פרויקט גמר

SmartRecipe AI — מחולל מתכונים חכם

SmartRecipe AI — A Full-Stack DevOps Project



Docker Compose · GitHub Actions · Prometheus · Grafana · Hetzner Cloud

שם הפרויקט	SmartRecipe AI — מחולל מתכונים חכם
שם הסטודנט	Gad Rapaport — גד רפפורט
תעודת זהות	214871915
מכללה	אורט סינגלובסקי — תכנית מיגו
מגמת לימוד	דבאופס (DevOps)
שם המנחה	שי אדרי
תאריך הגשה	09.07.2026

תוכן עניינים

1	רקע — DevOps מהו	3
2	מונולית לעומת מיקרו-שירותים	4
3	מטרת הפרויקט	5
4	סקירת מצב קיים בשוק	6
5	מה הפרויקט מחדש ומשפר	7
6	דרישות מערכת ופונקציונליות	8
7	בעיות צפויות ופתרונות	9
8	טכנולוגיות בשימוש	10
9	תיאור הפרויקט וארכיטקטורה כללית	11
10	מהכלל אל הפרט — שכבות ואבטחת הרשת	12
11	שכבת ה-Frontend — Nginx	13
12	שכבת ה-Backend — Python Flask	14
13	שכבת ה-SQL — MySQL	15
14	אינטגרציית AI — Gemini	16
15	אריזה בדוקר — Docker	17
16	הרצת השירותים עם Docker Compose	18
17	תהליך ה-CI/CD — GitHub Actions	19
18	עקרונות Git ו-GitOps בפרויקט	20
19	תשתית הענן — Hetzner	21
20	ניטור — Prometheus ו-Grafana	22
21	אבטחת הפייפליין — Secrets	23
22	בדיקות — pytest	24
23	מבנה תיקיות הפרויקט	25
24	שאלות צפויות מהבוחן	26

רקע — DevOps מהו

DevOps היא גישת פיתוח, תפעול וניהול תוכנה הדוגלת בביזור ופיצול תהליכים ושירותים, הפשטת החומרה מן התוכנה והנגשתה, פיתוח ושחרור מהיר של גרסאות, ואוטומציה בעזרת כלים ושיטות עבודה חדשניות תוך ניצול ושילוב תשתיות ענן.

המילה "DevOps" היא שילוב של שתי מילים: **Development** (פיתוח) ו-**Operations** (תפעול). בעולם המסורתי, צוותי הפיתוח וצוותי התפעול עבדו בנפרד, כאשר כל צד לא הכיר לעומק את עבודת הצד השני — המפתח לא הכיר את השרת, ואנשי התפעול לא הבינו את הקוד. גישת ה-DevOps פורצת את המחיצה הזו וגורמת לשני הצוותים לעבוד יחד, לחלוק אחריות ולהפוך לאוטומטי כל מה שניתן.

המטרה המרכזית

להוציא גרסאות תוכנה חדשות לייצור — מהר, בבטחה, ובאופן אוטומטי לחלוטין.

Plan → Code → Build → Test → Release → Deploy → Operate → Monitor
└────────── continuous loop ─────────┘

פרויקט זה נבנה כדי להדגים לולאה זו במלואה — מכתיבת שורת קוד ראשונה ועד להרצת האפליקציה בשרת ייצור בענן, כשכל שלב בדרך מבוקר, נבדק ואוטומטי.

מונוליט לעומת מיקרו-שירותים

גישת המונוליט (Monolith)

פיתוח בחבילה אחת גדולה המכילה את כל הספריות, הרכיבים והשירותים. התוצאה היא קוד ענקי שצוות התפעול מקבל בכל גרסה חדשה — כשהוא מבין בחומרה אך לא בתוכנה שבתוכה.

- פיתוח מסורבל — כל הרכיבים חייבים להיכתב באותה שפה.
- איתור באגים קשה עקב גודל הקוד.
- סקיילינג של רכיב אחד מחייב שדרוג של כל המערכת — בזבז משאבים.
- קשיי התאמה לפלטפורמות שונות.

גישת המיקרו-שירותים (Microservices)

כל רכיב ושירות הוא תוכנה קטנה ועצמאית, המחוברת לאחרים דרך ממשקי API. **יתרונות:** כל שירות בשפה משלו, סקיילינג עצמאי, איתור בעיות פשוט יותר ופריסה עצמאית. **חסרונות:** מורכבות תפעולית גבוהה, צורך בכלי Orchestration, יותר נקודות כשל פוטנציאליות ועלות הקמה גבוהה יותר.

הבחירה בפרויקט הנוכחי — ארכיטקטורה היברידית

הפרדה לשירותים נפרדים (Frontend, Backend, Database, Monitoring) שכל אחד רץ בקונטיינר Docker נפרד, אך הלוגיקה הפנימית של ה-Backend נכתבה כמונוליט מפושט. זו הגישה האופטימלית לפרויקט בסדר גודל זה — מורכבות ניהולית נמוכה, לצד הפרדה ברורה בין שכבות.

מטרת הפרויקט

הפרויקט מציג את תורת ה-DevOps דרך הדמיה מלאה של תהליך פיתוח תוכנה, פריסתה והרצתה, תוך שימוש בכמה מהטכנולוגיות המובילות כיום, והצגת המערך כולו על גבי שרת ענן אמיתי.

הפרויקט משלב שלושה צירים:

- **אפליקציית ווב שימושית** — מחולל מתכונים המשתמש במודל השפה Gemini של Google.
- **מערך DevOps מלא** — מ-commit ועד deployment אוטומטי לחלוטין.
- **ניטור ובקרה** — Grafana ו-Prometheus לניטור ביצועים בזמן אמת.

סקירת מצב קיים בשוק

יותר ויותר חברות עוברות כיום להטמעת תורת ה-DevOps ברמות שונות. קמות חברות ייעוץ ותפעול המתמחות אך ורק בתחום, וישנן חברות רבות המשווקות ומתפעלות שירותי אורקסטריציה, ניהול ותפעול קונטיינרים.

במקביל, בעידן ה-AI הגנרטיבי, מודלי שפה גדולים (LLM) כמו Gemini של Google ו-GPT של OpenAI הופכים לחלק אינטגרלי ממוצרי תוכנה. היכולת לשלב AI בתוך פייפליין DevOps רגיל — כולל בדיקות, קונטיינריזציה ו-CI/CD — היא כישור מבוקש בשוק העבודה כיום.

מה הפרויקט מחדש ומשפר

- **שילוב AI בפייפליין DevOps** — הפרויקט מדגים כיצד לשלב קריאות לממשקי (Gemini API) AI בתוך אפליקציה המנוהלת ב-CI/CD מלא.
- **Git כמקור האמת** — הפרויקט מיישם עקרונות GitOps ברמת הקוד: כל שינוי עובר דרך Git ומפעיל את הפייפליין האוטומטי.
- **אוטומציית תשתית** — סקריפטי Bash לאוטומציה מלאה של הכנת השרת ופריסת האפליקציה.
- **Monitoring מובנה מהיום הראשון** — Prometheus ו-Grafana משולבים עם dashboard מוגדר מראש.
- **ניהול סודות נכון** — GitHub Secrets ומשתני סביבה לניהול פריטים רגישים.

דרישות מערכת ופונקציונליות

דרישות לסביבת הרצה מקומית

- Docker Desktop (גרסה 24+) עם Docker Compose v2.
- Git.
- מפתח Gemini API חינוכי מ- `aistudio.google.com/app/apikey`.

דרישות לסביבת ייצור

- שרת Hetzner Cloud עם Ubuntu 22.04 (VPS).
- חשבון Docker Hub לאחסון images.
- חשבון GitHub לניהול קוד ו-CI/CD.
- מפתח SSH מוגדר לשרת.

פונקציונליות האפליקציה

- הזנת רשימת מרכיבים וקבלת מתכון שלם מ-AI.
- שמירה, עדכון ומחיקה של מתכונים.
- פילטור מתכונים לפי סוג מטבח.
- קבלת הצעות שינוי למתכון קיים מ-AI.
- ממשק ויזואלי נוח ונגיש.

בעיות צפויות ופתרונות

בעיה

בעת עדכון ושינוי קבצי המקור עלולים להיווצר באגים, ועם קצב עדכונים מהיר, האפליקציה עלולה ליפול בסביבת הייצור.

פתרון

הרצת בדיקות תוכנה (pytest) על כל שינוי בקוד. הפייפליין נעצר אוטומטית אם בדיקה נכשלת, ונשלח מייל התראה מיידי.

בעיה

תלות בין שירותים — Flask עלול לעלות לפני ש-MYSQL מוכן לקבל חיבורים.

פתרון

Docker healthcheck על MySQL בשילוב `depends_on: condition: service_healthy` מבטיחים שה-Backend עולה רק לאחר שה-DB מוכן לחלוטין.

בעיה

חשיפת סודות (API keys, סיסמאות) בתוך קוד המקור.

פתרון

שימוש במשתני סביבה, קובץ `.env`, ב-`.gitignore`, ו-GitHub Secrets לפייפליין. אף סוד לא מופיע בקוד המקור.

בעיה

פריצה לאחד הרכיבים עלולה לאפשר גישה למערכת כולה.

פתרון

רק Nginx חשוף לאינטרנט (פורט 80). Flask ו-MYSQL נמצאים ברשת Docker פנימית בלבד ואינם נגישים ישירות מהרשת החיצונית.

טכנולוגיות בשימוש

קונטיינר (Container) Image-I

קונטיינר הוא יחידה המדמה סביבה נפרדת וסטרילית במחשב — מעין "מחשב בתוך מחשב". מה שרץ בתוכו אינו מודע לתהליכים הרצים מחוצה לו. בניגוד למכונה וירטואלית, קונטיינר חולק את ליבת מערכת ההפעלה (Kernel) עם המחשב המארח, ולכן צורך פחות משאבים ויעיל יותר. ה-Image הוא מערכת הקבצים הפועלת בתוך הקונטיינר — כל הקבצים הנדרשים, מרמת מערכת ההפעלה ועד לתהליך עצמו. ניתן לדמות Image לכוון הקשיח של הקונטיינר.

Registry — Docker Hub

מקום אחסון ל-images, מקוטלג לפי שם ה-image וגרסה. בפרויקט נעשה שימוש ב-Docker Hub כרגיסטרי. כל push לענף main מעלה image חדש עם שני תגיות: `:latest` ו-`:sha-<hash>`, לצורך אפשרות Rollback מדויקת.

Docker Docker Compose-I

Docker הוא ממשק ליצירה, ניהול, החלפה ומחיקה של קונטיינרים. מאפשר יצירת רשתות וירטואליות לחיבור קונטיינרים, חיבור לרשת המחשב המארח, והזרמת מידע לקונטיינרים באמצעות Docker Volumes. Compose הוא כלי לניהול מספר קונטיינרים במקביל דרך קובץ YAML אחד. בפרויקט זה ה-Compose מנהל 5 שירותים: Grafana ו-Nginx, Flask, MySQL, Prometheus — כולל רשתות, healthchecks, Volumes וסדר הפעלה.

Git ו-GitHub

Git — מערכת מקומית לשמירת קבצי מקור, היסטוריה וגרסאות. **GitHub** — שירות ציבורי הפועל באינטגרציה מלאה עם Git, ומהווה את הטריגר לתהליך ה-CI/CD: על כל שינוי בקוד, GitHub Actions מופעל אוטומטית.

GitHub Actions

כלי CI/CD המשולב בגיטהאב, המחליף כלים כמו Jenkins. על כל עדכון קבצי מקור, ה-Workflow מופעל, בונה את התוצר הסופי, מריץ בדיקות ומבצע עדכון גרסה ופריסה אוטומטית לשרת.

Nginx

שרת Frontend ו-Reverse Proxy המאפשר אחסון קבצי ה-Frontend של האפליקציה, ובכך מוריד עומס מה-Backend ומספק שכבת הגנה נוספת. בפרויקט ממלא שני תפקידים: הגשת קבצי HTML/CSS/JS סטטיים, וניתוב בקשות `/api/*` לשרת Flask הפנימי.

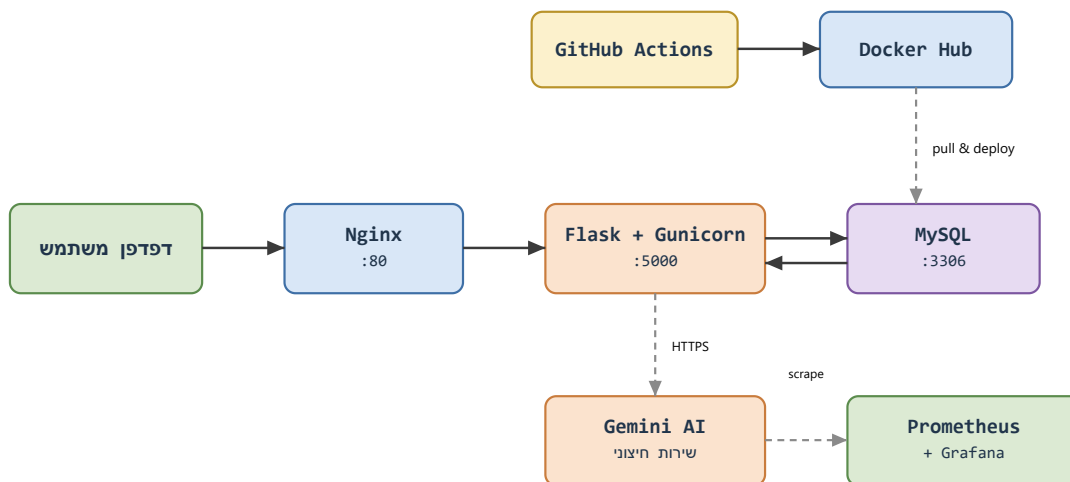
תיאור הפרויקט וארכיטקטורה כללית

הפרויקט הוא מודל DevOps המזגים תהליך מלא מרמת כתיבת קוד ועד פריסה בייצור. לשם כך נבנתה אפליקציית ווב שימושית — **SmartRecipe AI**. המשתמש מכניס מרכיבים שיש לו בבית (עגבנייה, עוף, שום...) ו-Gemini AI מייצר מתכון מלא עם הוראות הכנה, זמן הכנה, מידע תזונתי וסוג מטבח. המתכונים נשמרים ב-MYSQL ומוצגים ברשימה עם אפשרות דירוג ומחיקה.

המערך האוטומטי: על כל שינוי קוד שנדחף לגיטהאב מתחיל תהליך CI/CD אוטומטי — בדיקות pytest מול MySQL אמיתי, בנייה והעלאת Docker images ל-Docker Hub, ופריסה אוטומטית לשרת ה-Hetzner. המערכת כולה אוטומטית: לאחר ה-push אין מגע יד אדם עד שהגרסה החדשה עולה בשרת הייצור.

התרשים הבא מציג את רכיבי המערכת המרכזיים ואת הקשרים ביניהם — הן זרימת המשתמש והן זרימת הפריסה האוטומטית.

SmartRecipe AI — ארכיטקטורה כללית



(ניטור / AI / deploy) קו רציף = בקשת משתמש בזמן אמת | קו מקוקו = תהליך רקע

איור 1 — ארכיטקטורה כללית של המערכת

התרשים מדגיש שהמשתמש עובד תמיד מול Nginx בלבד, בעוד ש-MySQL, Flask ו-Gemini AI פועלים מאחורי שכבת הכניסה. תהליך הפריסה (חץ מקוקו מ-Docker Hub) פועל באופן עצמאי ומופעל על ידי GitHub Actions, ללא תלות בזמינות המשתמשים באתר.

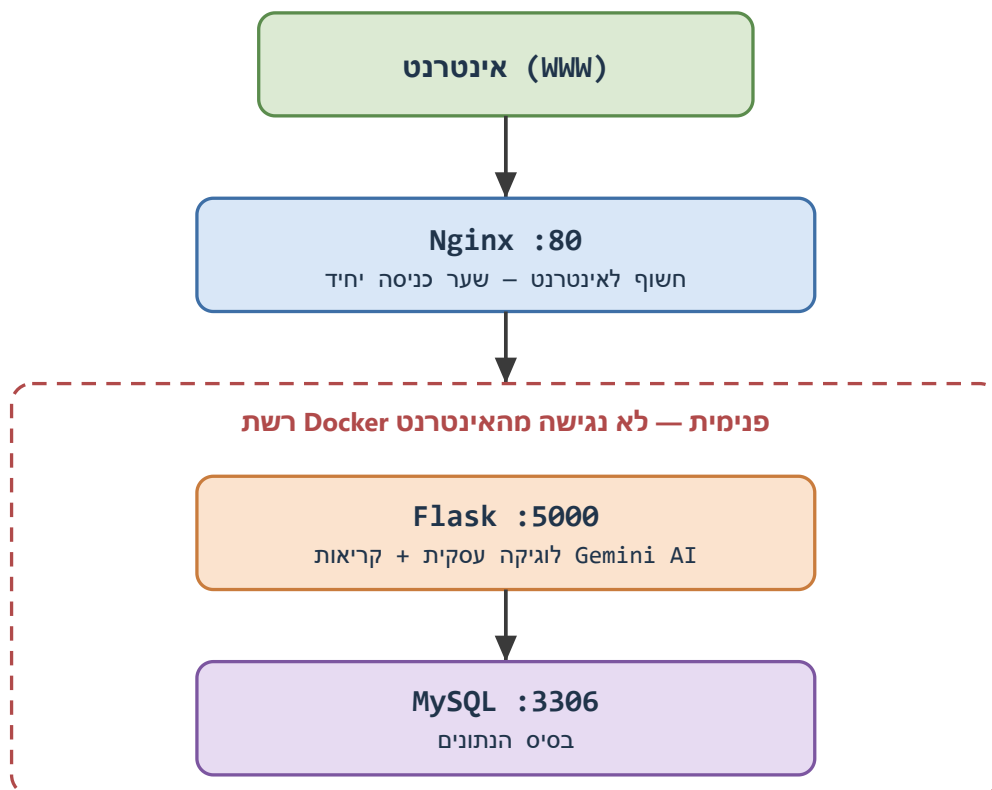
מהכלל אל הפרט — שכבות המערכת ואבטחת הרשת

האפליקציה שנכתבה היא מחולל מתכונים חכם (SmartRecipe AI), שבו המשתמש יכול לגשת לאתר, להכניס רשימת מרכיבים ולקבל מתכון מפורט בעזרת בינה מלאכותית. המתכונים נשמרים, ניתן לצפות בהם, לדרג אותם ולמחוק אותם. הניתוב באפליקציה מתבצע בממשק REST API.

חלקי האפליקציה

- **בסיס נתונים — MySQL:** מאחסן את כל המתכונים שהמשתמש שמר, כולל כותרת, מרכיבים, הוראות הכנה, מידע תזונתי, סוג מטבח, זמן הכנה, דירוג ותאריך יצירה.
- **Backend — Python Flask:** מנוע האפליקציה. מעבד בקשות HTTP, מתקשר עם ה-DB, קורא ל-Gemini AI ומחזיר תשובות JSON.
- **Frontend — Nginx + HTML/JS:** שרת קדמי בו מאוחסנים קבצי האתר, המשמש גם כ-Reverse Proxy לבאקאנד ואחראי על הניתוב וההגנה על שרת Flask.

הפרונט הוא **החלק היחיד החשוף לאינטרנט**. Flask ו-MYSQL נמצאים מאחורי Nginx, ברשת Docker פנימית בלבד.



שכבת ה-Nginx — Frontend

תפקידי Nginx בפרויקט

File Server: Nginx מגיש ישירות את קבצי ה-CSS, HTML ו-JavaScript לדפדפן המשתמש ללא מגע של Flask. זה חוסך עומס משרת ה-Python ומאפשר ל-Flask להתמקד בלוגיקה העסקית בלבד.

Reverse Proxy: כל בקשה שמגיעה ל- `/api/*` מנותבת על ידי Nginx לשרת Flask הפנימי:

```
location /api/ {
    proxy_pass http://backend:5000;
    proxy_read_timeout 120s;
}
```

ה-`timeout` של 120 שניות חשוב — קריאות ל-Gemini AI עשויות לקחת עד 15–30 שניות.

Load Balancing: בלוק ה- `upstream` ב-`nginx.conf` מאפשר הוספת מספר `instances` של Flask בעתיד:

```
upstream backend {
    server backend:5000;
    # server backend2:5000; # optional – add more instances
}
```

Security Headers: Nginx מוסיף headers שמגנים מפני XSS, Clickjacking ו-MIME sniffing.

ממשק המשתמש

ממשק המשתמש נבנה כ-Single Page Application (SPA) בוויליה JavaScript, ללא frameworks. כולל:

- **Tag Input** — הזנת מרכיבים כתגיות עם Enter.
- **AI Generator Panel** — הצגת המתכון שנוצר על ידי Gemini.
- **Recipe List** — רשימה ממוספרת עם pagination.
- **Modal** — הצגת פרטי מתכון מלאים.

שכבת ה-Python Flask Backend

App Factory Pattern

```
def create_app(config=None):
    app = Flask(__name__)
    db.init_app(app)
    migrate.init_app(app, db)
    metrics.init_app(app)    # Prometheus
    CORS(app)
    app.register_blueprint(recipes_bp, url_prefix="/api")
    app.register_blueprint(health_bp)
    return app
```

ה-App Factory Pattern מאפשר יצירת instances שונים של האפליקציה (dev, test, prod) — חיוני לבדיקות. הפונקציה `create_app()` מקבלת קונפיגורציה ומחזירה instance מוכן.

Blueprints — ניתוב מודולרי

תיאור	קידומת URL	Blueprint
health /-ו (index)	/	health
כל פעולות ה-AI + CRUD	/api	recipes

REST API Endpoints

שכבת ה-MYSQL — SQL

מבנה הטבלה

```
CREATE TABLE recipes (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  title       VARCHAR(255) NOT NULL,  
  ingredients  TEXT NOT NULL,  
  instructions TEXT NOT NULL,  
  nutritional_info TEXT,  
  cuisine_type VARCHAR(100),  
  prep_time_minutes INT,  
  rating      FLOAT DEFAULT 0.0,  
  created_at  DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

אינטגרציית Gemini — AI

Prompt Engineering

המפתח לקבלת תגובה מובנית ועקבית מ-Gemini הוא ה-Prompt:

```
prompt = f"""You are a professional chef and nutritionist.
Generate a detailed recipe using primarily these ingredients: {ingredients_str}.

Respond ONLY with a valid JSON object (no markdown) in this exact structure:
{{
  "title": "Recipe Name",
  "ingredients": "Full ingredient list with quantities",
  "instructions": "Step-by-step cooking instructions",
  "nutritional_info": "Calories, protein, carbs, fat per serving",
  "cuisine_type": "e.g. Italian, Asian",
  "prep_time_minutes": 30
}}"""
```

עקרונות ה-Prompt:

1. **תפקיד ברור** — "Professional chef and nutritionist".
2. **פורמט מוגדר** — JSON בלבד, ללא markdown.
3. **מבנה מדויק** — מניעת תגובות לא עקביות.
4. **Fallback** — ניקוי אוטומטי של backticks אם Gemini מוסיף אותם.

טיפול בשגיאות Gemini

```
try:
    recipe_data = generate_recipe(ingredients, preferences)
except Exception as e:
    return jsonify({"error": f"AI generation failed: {str(e)}"}), 500
```

במקרה של כשל (rate limit, timeout, תקלת רשת) מוחזרת הודעת שגיאה ידידותית ב-HTTP 500 עם הסבר. השגיאה נרשמת ב-logs של Gunicorn לצורך תחקור עתידי. המשתמש מוזמן לנסות שוב — אין Fallback למתכון מוכן מראש, כיוון שאפליקציה מסוג זה תלויה מטבעה ב-AI.

אבטחת ה-API Key

אריזה בדוקר — Docker

Dockerfile — Backend

```
FROM python:3.12-slim          # small base image
WORKDIR /app
RUN apt-get install -y \
    default-libmysqlclient-dev gcc pkg-config
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt # cache layer
COPY . .
CMD ["gunicorn", "--bind", "0.0.0.0:5000", \
    "--workers", "4", "--timeout", "120", "wsgi:app"]
```

Dockerfile — Frontend

```
FROM nginx:1.27-alpine          # very small base image
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY html/ /usr/share/nginx/html/
EXPOSE 80
```

היתרון המרכזי של Docker image הוא עקביות: אותו image שנבנה ונבדק ב-GitHub Actions הוא בדיוק ה-image שנשלח ל-Docker Hub ונמשך לשרת הייצור. כך מצטמצם כמעט לחלוטין הפער בין סביבת הבדיקה לסביבת ההרצה.

הרצת השירותים עם Docker Compose

docker-compose.yml — כל השירותים

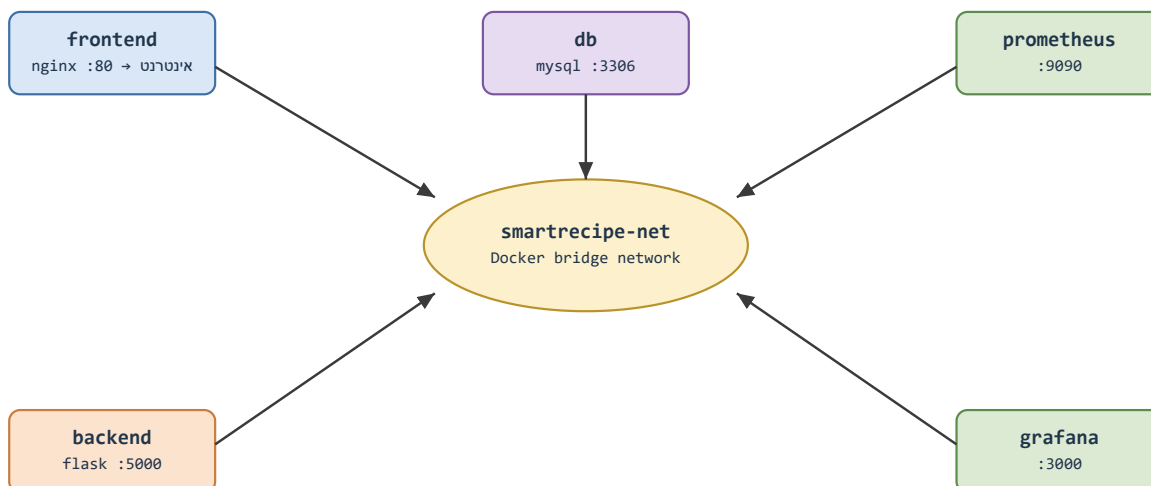
```
services:
  db:          # MySQL 8.0 + healthcheck + volume
  backend:     # Flask, depends_on: db (healthy)
  frontend:    # Nginx, port 80 → world
  prometheus:  # metrics, port 9090
  grafana:     # dashboard, port 3000
```

Override — docker-compose.prod.yml לייצור

```
services:
  backend:
    image: ${DOCKER_HUB_USERNAME}/smartrecipe-backend:${IMAGE_TAG}
    build: ~ # disables local build – pulls from Docker Hub instead
  frontend:
    image: ${DOCKER_HUB_USERNAME}/smartrecipe-frontend:${IMAGE_TAG}
    build: ~
```

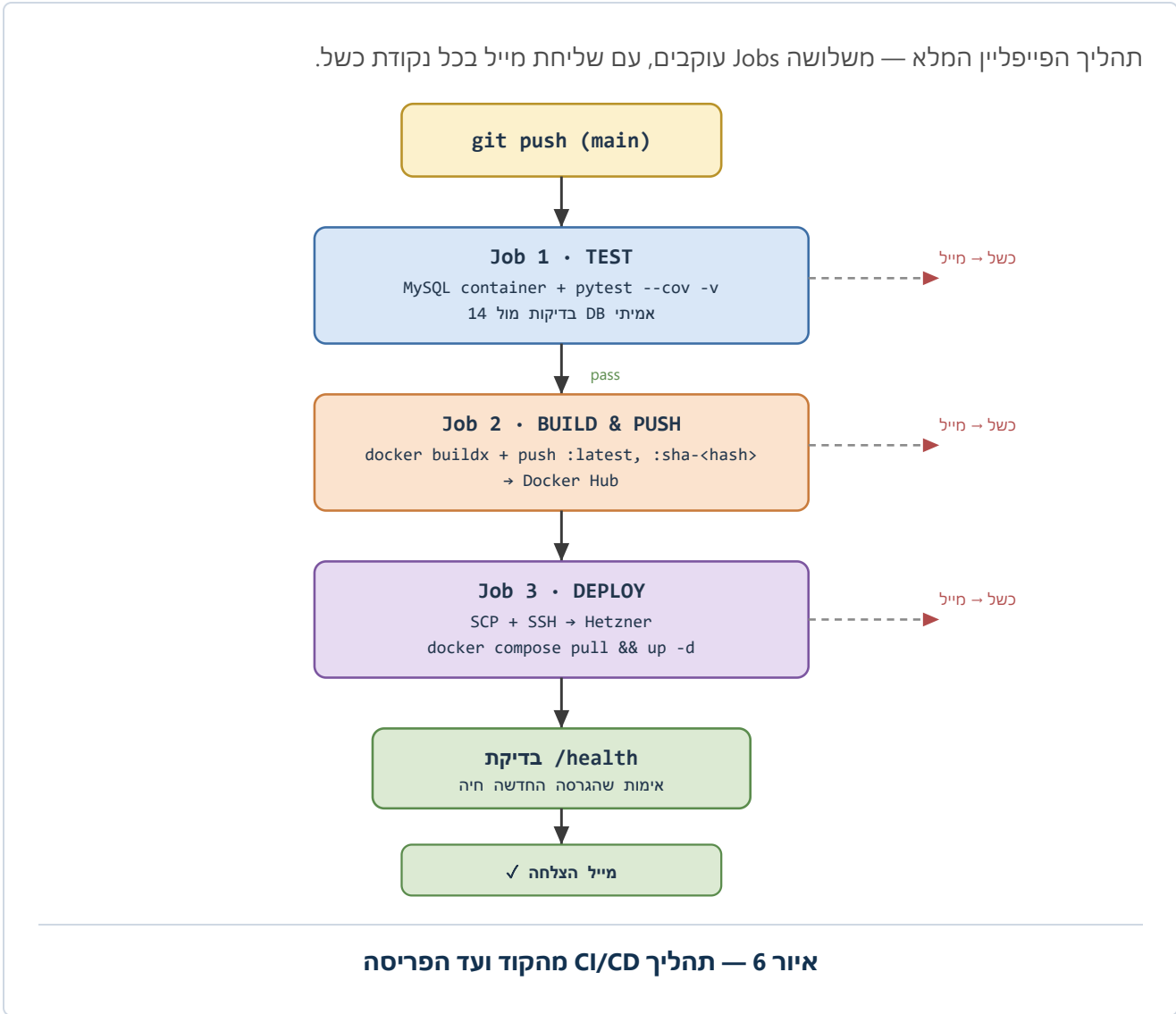
בסביבת ייצור, תמונות מורדות מ-Docker Hub בלבד — אין בנייה מקומית (local build).

חמשת השירותים המנוהלים על ידי Docker Compose, מחוברים דרך רשת פנימית משותפת.



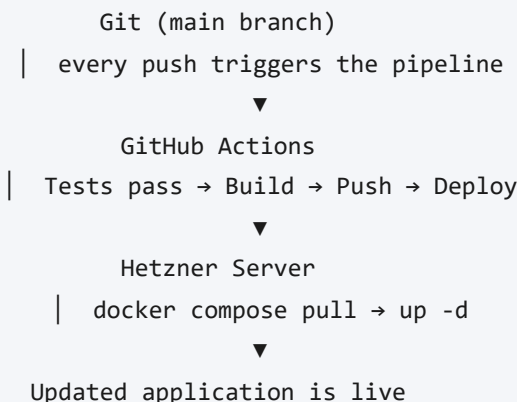
תהליך ה-GitHub Actions — CI/CD

על כל `git push` לענף `main` מופעל ה-Workflow, אוטומטית, לפי ההגדרה ב- `.github/workflows/ci-cd.yml`.



עקרונות Git ו-GitOps בפרויקט

הפרויקט מיישם עקרונות GitOps ברמת הקוד והקונפיגורציה: **כל שינוי עובר דרך Git ומפעיל את הפייפליין האוטומטי**. חשוב לציין שהמונח "GitOps מלא" בתעשייה מתייחס בדרך כלל למערכות מבוססות Kubernetes עם כלי Reconciliation כמו ArgoCD או Flux, שם Kubernetes עצמו מסנכרן ברציפות את מצב הייצור עם המוגדר ב-Git. בפרויקט זה נעשה שימוש ב-GitHub Actions עם Docker Compose ו-SSH Deploy — קלאסי שמיישם את עקרון "Git כמקור האמת" ברמת הקוד.



עקרונות שיושמו בפרויקט

1. **הכל ב-Git** — קוד, Dockerfiles, docker-compose, nginx.conf, סקריפטי הכנת שרת.
2. **אין שינויים ידניים בשרת** — כל שינוי עובר דרך הפייפליין בלבד.
3. **Immutable Images** — כל deploy משתמש ב-image חדש עם sha tag ייחודי.
4. **Rollback פשוט** — שינוי `IMAGE_TAG` ל-sha קודם ו-push מפעיל פריסה של הגרסה הישנה.

תשתית הענן — Hetzner

הכנת השרת — hetzner_setup.sh

סקריפט Bash שמכין שרת Ubuntu 22.04 מאפס:

```
# 1. update packages
apt-get update && apt-get upgrade -y

# 2. install Docker from the official repository
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg --dearmor...
apt-get install docker-ce docker-ce-cli containerd.io

# 3. create a deploy user with Docker permissions
useradd -m -s /bin/bash deploy
usermod -aG docker deploy

# 4. firewall configuration (UFW – Security Groups)
ufw allow 80/tcp      # application
ufw allow 443/tcp     # HTTPS
ufw allow ssh        # management
ufw allow 3000/tcp    # Grafana
ufw allow 9090/tcp    # Prometheus

# 5. create directories for persistent data
mkdir -p /opt/smartrecipe/{mysql_data,prometheus_data,grafana_data}
```

Firewall — Security Groups

בדומה ל-Security Groups ב-AWS, UFW (Uncomplicated Firewall) מגדיר אילו פורטים פתוחים. רק פורטים הכרחיים פתוחים — פורט MySQL (3306) **סגור לחלוטין** לעולם החיצוני.

מדוע Hetzner ולא AWS?

Hetzner מציע VPS זול משמעותית (כ-2€–5€ לחודש, לעומת עשרות דולרים ב-AWS EC2). לפרויקט בסדר גודל זה, עם Docker Compose ואפליקציה אחת, אין צורך בשירותים המנוהלים של AWS — כל יכולות ה-DevOps (Docker, SSH, firewall) זמינות בשרת Ubuntu רגיל.

ניטור — Grafana ו-Prometheus

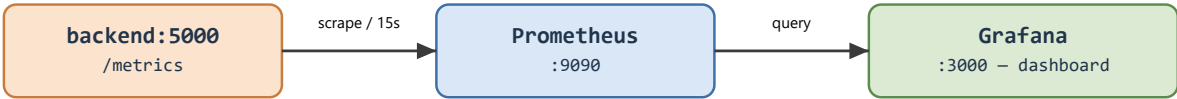
Prometheus — איסוף מדדים

Prometheus מתחבר כל 15 שניות ל- `/metrics` של Flask (דרך `prometheus-flask-exporter`) ושומר את הנתונים:

```
scrape_configs:
  - job_name: "smartrecipe-backend"
    static_configs:
      - targets: ["backend:5000"]
    metrics_path: "/metrics"
```

ממד	תיאור
<code>flask_http_request_total</code>	סך הבקשות לפי <code>method, path, status</code>
<code>flask_http_request_duration_seconds</code>	זמני תגובה (histogram)
<code>flask_http_request_exceptions_total</code>	בקשות שגרמו ל-exception

זרימת הנתונים ממדדי האפליקציה ועד לתצוגה הגרפית.



איור 7 — זרימת ניטור Prometheus ו-Grafana

Grafana — ויזואליזציה

Dashboard מוכן מראש (Provisioning — Configuration as Code):

```
# datasources/prometheus.yml
datasources:
  - name: Prometheus
```

אבטחת הפייפליין — Secrets

GitHub Secrets

שימוש	Secret
שם משתמש Docker Hub	DOCKER_HUB_USERNAME
Access Token (לא סיסמה)	DOCKER_HUB_TOKEN
IP השרת	HETZNER_HOST
משתמש SSH	HETZNER_USER
מפתח SSH פרטי	HETZNER_SSH_KEY
מפתח Gemini AI	GEMINI_API_KEY
מייל לשליחת התראות	MAIL_USERNAME
Gmail App Password של	MAIL_PASSWORD
כתובת לקבלת התראות	MAIL_RECIPIENT

Docker Secrets / Environment Variables

```
# docker-compose.yml – values come from .env
environment:
  GEMINI_API_KEY: ${GEMINI_API_KEY}
  MYSQL_PASSWORD: ${MYSQL_PASSWORD}
  SECRET_KEY: ${SECRET_KEY}
```

קובץ `.env` נמצא ב- `.gitignore` ואינו מועלה ל-Git.

עקרונות אבטחה נוספים

1. **MySQL לא חשוף לאינטרנט** — רק ברשת Docker הפנימית.
2. **Flask לא חשוף לאינטרנט** — רק Nginx.
3. **HTTPS** — כרגע האתר פועל על HTTP בלבד. בסביבת ייצור ניתן להוסיף Let's Encrypt באמצעות Certbot ל-Nginx ללא שינוי בקוד האפליקציה.
4. **SQLAlchemy ORM** — הגנה מובנית מפני SQL Injection.

בדיקות — pytest

הבדיקות מחולקות לשתי קטגוריות, ושתיים רצות מול **MySQL אמיתי** (לא mock) — כפי שנדרש באפיון: "טסט שבדוק הכנסה/הוצאה מה-DB ועובר כמובן דרך השרת."

test_db.py — בדיקות הכנסה/הוצאה מה-DB

בדיקה	מה נבדק
test_insert_recipe	הכנסת רשומה ל-DB ווידוא קבלת ID
test_retrieve_recipe	שמירה ושליפה לפי שדה
test_update_recipe	עדכון שדה ווידוא הנתונים החדשים
test_delete_recipe	מחיקה ווידוא שהרשומה נמחקה
test_recipe_to_dict	serialization לפורמט JSON

test_api.py — בדיקות API דרך השרת

בדיקה	מה נבדק
test_health_endpoint	GET /health מחזיר 200
test_list_recipes_empty	GET /api/recipes מחזיר JSON תקין
test_save_recipe	POST יוצר רשומה ומחזיר 201
test_get_recipe	GET /:id מחזיר את הרשומה הנכונה
test_update_recipe	PUT מעדכן שדה
test_delete_recipe	DELETE מוחק ומחזיר 404 לאחר מכן
test_save_recipe_missing_field	POST ללא שדה חובה → 400
test_generate_recipe_missing_ingredients	POST ללא מרכיבים → 400

conftest.py — Fixtures

מבנה תיקיות הפרויקט

מבנה זה מציג את קבצי הפרויקט המרכזיים: תיקיית `backend/` מכילה את קוד אפליקציית Flask לפי App Factory Pattern; תיקיית `frontend/` כוללת את קבצי ה-Nginx וה-HTML/JS הסטטיים; תיקיית `infrastructure/` מכילה את סקריפטי הכנת השרת והפריסה; תיקיית `monitoring/` כוללת את הגדרות Prometheus וגרפנה; ותיקיית `.github/workflows` מכילה את תהליך ה-CI/CD כולו.

```
smartrecipe-ai/
├── .env.example
├── .gitignore
├── docker-compose.yml
├── docker-compose.prod.yml
├── PROJECT_BOOK_HE.md
├── README.md / README-he.md
├── .github/
│   └── workflows/
│       └── ci-cd.yml                # Test → Build&Push → Deploy
├── backend/
│   ├── Dockerfile
│   ├── wsgi.py
│   ├── requirements.txt
│   ├── pytest.ini
│   └── app/
│       ├── __init__.py            # create_app() – App Factory
│       ├── models.py              # Recipe model
│       └── routes/
│           ├── health.py
│           └── recipes.py         # REST API + Gemini AI
│       └── services/
│           └── gemini_service.py  # Prompt engineering
│   └── tests/
│       ├── conftest.py
│       ├── test_api.py
│       └── test_db.py
├── frontend/
│   ├── Dockerfile
│   ├── nginx.conf                 # reverse proxy + static files
│   └── html/index.html            # SPA – Vanilla JS
├── infrastructure/
│   ├── deploy.sh                  # runs on the server on every deployment
│   ├── hetzner_setup.sh          # prepares a fresh server from scratch
│   └── mysql-init.sql
└── monitoring/
    ├── prometheus/prometheus.yml
    └── grafana/provisioning/
        ├── datasources/prometheus.yml
        └── dashboards/smartrecipe.json
```

שאלות צפויות מהבוחן

למה Hetzner ולא AWS?

Hetzner מציע VPS זול משמעותית (2–5€/חודש לעומת עשרות דולרים ב-AWS EC2). לפרויקט בסדר גודל זה, עם Docker Compose ואפליקציה אחת, אין צורך בשירותים המנוהלים של AWS. כל יכולות ה-DevOps (Docker, SSH, firewall) זמינות בשרת Ubuntu רגיל.

למה Docker Compose ולא Kubernetes?

Kubernetes מתאים למערכות עם עשרות מיקרו-שירותים הזקוקים ל-rolling-update, auto-scaling, self-healing. לפרויקט זה עם 5 שירותים, Docker Compose פשוט יותר לתפעול, קל יותר לדיבאג ומספיק לגמרי.

למה Flask ולא FastAPI?

Flask הוא הפריימוורק הנפוץ ביותר בקורסי Python ומוכר יותר לבוחנים. FastAPI מהיר יותר ותומך ב-async, אך Flask מספיק לאפליקציה זו ובעל ecosystem עשיר (Flask-SQLAlchemy, Flask-Migrate).

למה MySQL ולא PostgreSQL?

MySQL מוכר יותר בקורסים ובתעשייה לפרויקטים בסדר גודל זה. לשניהם ביצועים דומים לאפליקציה זו. אם היה נדרש שימוש ב-JSON columns מתקדם או Full-Text Search מורכב, PostgreSQL היה עדיף.

למה GitHub Actions ולא Jenkins?

GitHub Actions משולב ישירות בגיטהאב — אין צורך בשרת Jenkins נפרד. עבור פרויקט בודד, Jenkins מוסיף תשתית לתחזוקה ללא יתרון משמעותי. GitHub Actions חינוכי ומספיק בעוצמתו.

איך מתבצע Rollback?

לכל commit יש Docker image עם תג `:sha-<hash>` ייחודי. לחזרה לגרסה קודמת: משנים את `IMAGE_TAG` ב-`.env` בשרת לגרסה הרצויה ומריצים `docker compose up -d`. אפשר גם לבצע `git revert` ו-push — הפייפליין יפרס את הגרסה הישנה אוטומטית.

איך נשמרים הסודות?

שלוש שכבות: (1) GitHub Secrets לפייפליין — מוצפנים ולא נחשפים ב-logs; (2) `.env` בשרת הייצור — לא ב-Git, הרשאות 600; (3) בקוד — רק דרך `os.getenv()`, לא hard-coded.

מה קורה אם Gemini לא זמין?

הבקשה נכשלת עם HTTP 500 ומוחזרת הודעת שגיאה ידידותית. השגיאה נרשמת ב-logs. האפליקציה ממשיכה לעבוד לגמרי — כל פעולות ה-CRUD של מתכונים אינן תלויות ב-Gemini.

איך הבדיקות מתחברות למסד הנתונים?

ב-GitHub Actions, CI מרים container של MySQL כ-service במכונה זמנית. הבדיקות מתחברות ל-`127.0.0.1:3307` — MySQL אמיתי, לא mock — דרך `TEST_DATABASE_URL`.

מה ההבדל בין Docker Volume לבין Backup?

Volume מגן מפני מחיקת נתונים כשה-container נעצר. אך אם הדיסק ניזוק, ה-volume נמחק, או מורץ `docker compose down -v` — הנתונים אבודים. גיבוי (mysqldump יומי) שומר עותק חיצוני ומאפשר שחזור. אחד לא מחליף את השני.

סיכום ושיפורים עתידיים

מה הושג בפרויקט

דרישה מהאפיון	מה בוצע	סטטוס
Flask backend	Flask 3.0 + Gunicorn, REST API מלא	✓
Nginx — Load Balancing / File Server	nginx.conf עם upstream + static files	✓
MySQL + חיבורי API	SQLAlchemy ORM + Flask-Migrate	✓
אריזה בדוקר (שרת, Nginx, DB)	5 containers ב-Docker Compose	✓
אחסון פריטים רגישים (Secrets)	GitHub Secrets + .env + .gitignore	✓
pytest — בדיקת הכנסה/הוצאה מה-DB	14 בדיקות מול MySQL אמיתי	✓
CI — הרצה בעדכון קוד	GitHub Actions trigger on push	✓
עדכון מייל אם נכשל	dawidd6/action-send-mail@v3	✓
העברת images ל-Docker Hub	Build & Push עם :sha: + latest	✓
פריסה בענן (Hetzner)	SSH deploy + docker compose up	✓
Bash script בהרצת המכונה	hetzner_setup.sh + deploy.sh	✓
Prometheus + Grafana	containers + provisioning מוכן	✓
AI integration	Gemini 1.5 Flash	✓

לקחים עיקריים

1. **CI/CD אוטומטי עובד** — כל שינוי עובר דרך pipeline → Git → ייצור, ללא שינויים ידניים.
2. **Real DB בבדיקות** — בדיקות המתחברות ל-MySQL אמיתי תופסות בעיות שמוק לא היה תופס.
3. **Environment Variables חיוניים** — אף סוד לא מוטמע בקוד, וניתן להחליף בכל סביבה.
4. **Healthchecks מונעים בעיות** — ללא Flask, healthcheck היה עולה לפני ש-MySQL מוכן ומתרוסק.
5. **Monitoring מהיום הראשון** — קל יותר לבנות ניטור בתחילת הפרויקט מאשר להוסיפו בהמשך.
6. **Docker cache** — סדר הפקודות ב-Dockerfile משפיע ישירות על זמן הבנייה.

כיוונים עתידיים

נספחים — פקודות שימושיות

תפקיד	פקודה
הרצת הפרויקט מקומית	<code>docker compose up --build</code>
הרצת סביבת production-style	<code>docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d</code>
משיכת images חדשים	<code>docker compose pull</code>
בדיקת זמינות	<code>curl http://localhost/health</code>
הרצת בדיקות עם כיסוי קוד	<code>pytest --cov=app -v</code>
ניהול migrations במסד הנתונים	<code>flask db migrate / upgrade</code>
צפייה בלוגים של האפליקציה	<code>docker logs --tail=120 smartrecipe-backend</code>
בדיקת image פעיל	<code>docker inspect smartrecipe-backend --format='{{.Config.Image}}'</code>
ניקוי images ישנים בשרת	<code>docker image prune -f</code>

Workflow

תפקיד	קובץ
Test → Build & Push → Deploy, כולל שליחת מיילים	<code>.github/workflows/ci-cd.yml</code>